

# 講義 23：期末整合練習——訂單 Header/Detail ( 授課順序：接在講義 28 之後，全課程最後一講；需要講義 21 的外鍵 / Search Help )

對應練習：ex23 | 答案物件：資料表 ZTR23\_ORDH + ZTR23\_ORDI + 程式 ZR\_TR23\_ORDERS

## 本講重點

- 貨真價實的 Header/Detail ( 單頭/單身 ) 資料模型：跟 SAP 標準的 VBAK/VBAP、EKKO/EKPO 同一種結構
- Domain 固定值清單 ( 跟講義 21 的區間值域是不同技巧 )
- 外鍵欄位本身就是 Key 的寫法
- LUW all-or-nothing 實地驗證**：Header+Detail 同一筆交易失敗時整包撤銷

## 1. 為什麼要有這一講：ex21 vs ex23 的差異

講義 21 已經教過 DDIC 外鍵、Check Table、Search Help，但那個「班級 - 學生」的關聯本質上是分類查詢：學生查自己屬於哪個班級，比較像「下拉選單」的性質。

本講的「訂單 - 明細」是真正的單頭/單身 ( **Header/Detail** ) 交易資料：一張訂單開出去，Header ( 訂單號、客戶、日期、狀態 ) 跟 Detail ( 每一行買了什麼、數量、單價 ) 必須同時存在，缺一不可——沒有明細的訂單頭沒有意義，沒有訂單頭的明細更是無效資料。這種「一組資料必須一起成功或一起失敗」的情境，才是 **LUW ( Logical Unit of Work )** 概念真正要解決的問題，講義 21 只講了觀念，這一講讓學生親眼看到。

## 2. 資料模型：ZTR23\_ORDH ( Header ) / ZTR23\_ORDI ( Detail )

| 表          | 角色     | Key                  | 說明             |
|------------|--------|----------------------|----------------|
| ZTR23_ORDH | Header | MANDT, ORDNO         | 訂單主檔：客戶、日期、狀態  |
| ZTR23_ORDI | Detail | MANDT, ORDNO, ITEMNO | 訂單明細：一個訂單多筆明細行 |

跟講義 21 的班級/學生比較：

|             | ex21 ( 班級 / 學生 )          | ex23 ( 訂單 / 明細 )                  |
|-------------|---------------------------|-----------------------------------|
| 關聯性質        | 分類查詢 ( 學生「屬於」哪個班級 )       | 交易資料 ( 訂單「包含」哪些明細 )               |
| 外鍵欄位是不是 Key | 不是 ( KLASSE 是一般欄位 )       | 是 ( ORDNO 同時是 Detail 的 Key 和外鍵 )  |
| 沒有這筆關聯合理嗎   | 合理 ( 學生可以還沒分班，KLASSE 空白 ) | 不合理 ( 明細一定要屬於某張訂單，不會有「沒有訂單號的明細」 ) |

第二點的差異在 DDL 上就看得出來：

```
@AbapCatalog.foreignKey.label : 'Check Against Order Header'
@AbapCatalog.foreignKey.screenCheck : true
key ordno : ztr23_ordno not null
  with foreign key [0..*,1] ztr23_ordh
    where mandt = ztr23_ordi.mandt
      and ordno = ztr23_ordi.ordno;
```

**ordno** 前面多了 **key**——外鍵欄位本身就是 **Detail** 表複合主鍵的一部分，這是 SAP 官方文件 **SPFLI** 外鍵到 **SCARR** 範例的原版寫法，比講義 21 的 **KLASSE** (非 Key 外鍵) 更貼近教科書。

### 3. Domain 固定值清單：跟講義 21 的區間是不同工具

**ZTR21\_SCORE** 用的是區間 (0~999，範圍內都合法)；本講 **ZTR23\_STATUS** 示範**固定值清單** (只有 **N/R/C** 三個值合法，中間的值都不合法)：

| Domain 技巧            | 適合情境                   | 本課範例                                             |
|----------------------|------------------------|--------------------------------------------------|
| 區間 (Interval)        | 數值有連續合法範圍              | <b>ZTR21_SCORE</b> : 0~999                       |
| 固定值清單 (Fixed Values) | 只有少數幾個離散合法值 (狀態、旗標、類別) | <b>ZTR23_STATUS</b> : N (新建) / R (已確認) / C (已取消) |

兩種都是「值域檢核」的一種，選哪個看資料的性質——有限選項用固定值清單，並附上說明文字 (Value Range 頁籤逐筆輸入)，SM30/畫面上會出現 F4 選單，比 Search Help 更輕量 (不用額外建 Search Help 物件)，但只適合「選項少又固定」的情境；選項多或會變動 (如班級、產品) 還是要用 Search Help + 外鍵。

### 4. 程式：LUW all-or-nothing 實地示範

這是本題的核心，程式碼重點片段：

```
INSERT ztr23_ordh FROM gs_ordh.           " Header INSERT 成功，sy-subrc = 0
INSERT ztr23_ordi FROM gs_ordi.           " 明細 001 INSERT 成功，sy-subrc = 0
INSERT ztr23_ordi FROM gs_ordi.           " 故意重複 INSERT 同一筆，sy-subrc = 4 (模擬失敗)
```

```
IF sy-subrc <> 0.
  ROLLBACK WORK.                           " 整個 LUW 撤銷——包含前面已經成功的 Header !
ENDIF.
```

```
SELECT SINGLE * FROM ztr23_ordh INTO gs_check WHERE ordno = 'ORD0002'.
" sy-subrc = 4 : 查無此訂單，Header 也不見了
```

**關鍵觀念**：**INSERT ztr23\_ordh** 那一行執行完 **sy-subrc = 0**，資料庫層面「當下」確實寫進去了，但只要整個交易還沒 **COMMIT WORK**，這筆寫入就只是**暫時性的**，**ROLLBACK WORK** 一執行，連這筆「本來成功」的 Header 也會一起消失。這才是 LUW 的完整意義：**COMMIT WORK** 之前，一切都還可以整包撤銷——不管中間有幾個 INSERT/UPDATE 各自回報成功。

這也解釋了為什麼 Header+Detail 這種交易資料，正式程式**絕對不能**每個 INSERT 後面都馬上 COMMIT WORK：如果第一個 INSERT 後面就 COMMIT，之後才發現第二個明細失敗，Header 已經真的、永久地寫進資料庫了，ROLLBACK 也救不回來，變成「只有訂單頭沒有明細」的髒資料。

## 5. 常見錯誤與陷阱

| 症狀                                               | 原因                                                                          |
|--------------------------------------------------|-----------------------------------------------------------------------------|
| Search Help Activate 失敗                          | Selection Method 表裡的欄位沒有 Data Element ( 講義 21 教訓， <b>CUSTOMER</b> 一定要有 DE ) |
| 以為 ROLLBACK WORK 只會撤銷「最後一個」INSERT                | 錯——LUW 是以「上一次 COMMIT/ROLLBACK 之後」的所有動作為單位整包處理，不分先後                          |
| Header+Detail 中間插了一個 COMMIT WORK                 | 一旦 COMMIT，前面的動作就永久生效，之後再 ROLLBACK 救不回已提交的部分——這是「交易邊界」設計錯誤，思考題 2 會碰到         |
| JOIN 的 ON 條件寫了 MANDT 或 ORDER BY 放在 INTO TABLE 後面 | 語法錯誤：client 欄位由編譯器自動處理； <b>ORDER BY</b> 要寫在 <b>INTO TABLE</b> 之前            |
| 外鍵設了 Screen Check 還是插入孤兒明細                       | 正常——DDIC 外鍵只擋畫面輸入，Open SQL 不受影響 ( 講義 21 重點，這題再驗證一次 )                        |

## 6. 課堂練習

完成 [ex23](#)：建立訂單 Header/Detail 兩張表 ( 含外鍵、固定值 Domain、Search Help )，寫程式驗證 LUW all-or-nothing、外鍵行為、JOIN 輸出。這是基礎課除 ex13 之外的第二個期末整合練習，完成後代表 DDIC 三層件、外鍵、Search Help、LUW 這條主線已經融會貫通。